AntiBot-GW/1.0

Reverse Proxy with Layered Anti-Automation Defenses

Implementation Report

Author: Pedro Tarrinho

Date: 2026-04-26

Target: sporting.cfsectraining.com

Stack: Python 3.13 / aiohttp / ngrok

Purpose: Defensive control evaluation — bypass-testing harness

Document version: 1.0

1. Executive Summary

This report describes **AntiBot-GW/1.0**, a reverse HTTP proxy implemented to sit in front of the CTF challenge `sporting.cfsectraining.com` for the purpose of evaluating anti-automation defenses. The proxy implements **six independent protection layers**, each addressing a distinct attack surface used by automated agents (CLI tools, scrapers, headless browsers, AI agents).

The objective is twofold: *(i)* demonstrate working defenses against common automated traffic patterns, and *(ii)* expose the deliberate weaknesses of each layer so they can be tested and bypassed in a controlled environment. This dual nature makes the proxy a learning artifact for offensive testing of defensive controls — every layer is wired to be inspected at runtime via internal endpoints (`/__status` , `/__xff`).

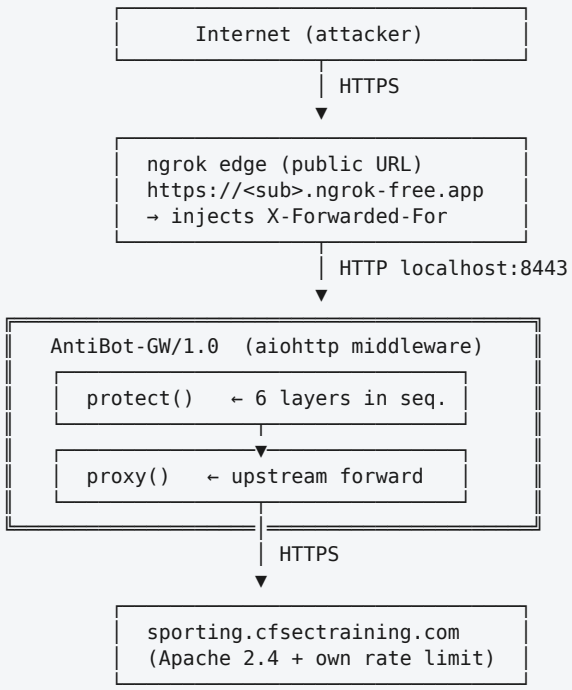
Implemented controls

#	Control	Trigger response	Description
1	Ban list	403	In-memory blocklist of IPs that previously hit a honeypot path.
2	Honeypot paths	403 + ban	Trap URLs (e.g. `/wp-admin`) that auto-ban any visitor for 1h.
3	UA blocklist	403	Substring match against a list of automation-tool User-Agents.
4	Rate limit	429	Per-IP token bucket; configurable burst and refill rate.
5	Behavioral check	403	Statistical analysis of inter-request timing (σ, μ).
6	Proof-of-Work	402	Hashcash-style puzzle gating sensitive endpoints.

All six layers were tested individually and end-to-end through an ngrok tunnel. Each layer functions as designed and each has a known bypass vector documented in §6.

2. Architecture

2.1 Deployment topology



2.2 Request lifecycle

```
Request  →  protect() middleware  →  6 layers
                                        ▼ (all pass)
                                   proxy() handler
                                        ▼
                                   upstream call
                                        ▼
                                    response back
```

2.3 In-memory state model

```python
@dataclass
class IpState:
    tokens: float           # current token-bucket capacity (max BURST)
    last_refill: float      # monotonic timestamp of last refill
    banned_until: float     # 0 if not banned, else absolute monotonic time
    request_times: deque    # last 50 request timestamps (for behavioral check)
    request_count: int      # cumulative count
    first_seen: float       # first appearance of this IP
```

State is keyed by client IP (resolved via the `get_ip()` helper), held in a `defaultdict` guarded by an `asyncio.Lock`, and is process-local — restarting the proxy clears all bans and counters.

3. Protection Layers

1 Ban List

Simple membership check against `ip_state[ip].banned_until`. If the value is greater than the current monotonic time, the request is rejected immediately, before any other check.

```
banned, remaining = await is_banned(ip)
if banned:
    return Response(status=403,
                    text=f"banned for {int(remaining)}s",
                    headers={"X-Reason": "banned"})
```

Default ban duration: 1h (3600s). Bans are populated by Layer 2 (honeypot).

2 Honeypot Paths

The proxy maintains a hard-coded set of paths that legitimate users never intentionally visit. Any request to these paths immediately bans the source IP for 1 hour.

```
HONEYPOT_PATHS = {
  "/wp-admin", "/wp-login.php", "/.env", "/.git/config",
  "/admin-backup", "/config.php.bak", "/phpinfo.php",
  "/xmlrpc.php", "/.aws/credentials", "/server-status",
  "/.well-known/admin", "/admin.php", "/phpmyadmin",
}

if request.path in HONEYPOT_PATHS:
    await ban(ip)
    return Response(status=403, text="banned (honeypot)",
                    headers={"X-Reason": "honeypot"})
```

Targets indiscriminate vulnerability scanners (nuclei, dirbuster, ffuf default wordlists). Real browsers never request these paths because they are not linked from the application HTML.

3 User-Agent Blocklist

Case-insensitive substring match against a curated list of automation fingerprints. Detects default User-Agents from CLI tools, scripting libraries, headless browsers, and AI/LLM crawlers.

```
UA_BLOCKLIST = (
  "curl/", "python-requests/", "httpx/", "go-http-client/",
  "wget/", "scrapy/", "java/", "okhttp/",
  "claude", "chatgpt", "gptbot", "anthropic",
  "selenium", "headless", "puppeteer", "playwright", "phantomjs",
)

ua_lower = ua.lower()
for blocked in UA_BLOCKLIST:
    if blocked in ua_lower:
        return Response(status=403,
                        body=json.dumps({"error":"user-agent blocked",
                                         "matched":blocked}),
                        headers={"X-Reason":"ua-blocked"})
```

4 Per-IP Rate Limit (Token Bucket)

Each IP gets a token bucket of `BURST` capacity, refilling at `REFILL` tokens per second. Each successful request consumes one token. When the bucket is empty, the proxy returns `429 Too Many Requests` with a calculated `Retry-After` header.

```
async def take_token(ip) -> tuple[bool, float, int]:
    s = ip_state[ip]
    n = now()
    elapsed = n - s.last_refill
    s.tokens = min(BURST, s.tokens + elapsed * REFILL)
    s.last_refill = n
    if s.tokens >= 1.0:
        s.tokens -= 1.0
        return True, 0.0, int(s.tokens)
    retry = (1.0 - s.tokens) / REFILL
    return False, retry, 0
```

Response on rate-limit hit:

```
HTTP/2 429 Too Many Requests
Retry-After: 1
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-Reason: rate-limit
Content-Type: application/json

{"error":"rate limit exceeded","retry_after":1}
```

5 Behavioral Check

Tracks the last 50 request timestamps per IP. Once 8 requests have been seen, the proxy computes the standard deviation σ and mean μ of inter-request intervals. If σ < 10 ms **and** μ < 1 s, the traffic is flagged as too regular for human origin.

```
async def behavioral_check(ip):
    s = ip_state[ip]
    if len(s.request_times) < BEHAVIOR_MAX_REGULAR:  # 8
        return False, ""
    recent = list(s.request_times)[-8:]
    intervals = [recent[i+1] - recent[i] for i in range(7)]
    μ = mean(intervals)
    σ = sqrt(mean((iv - μ)**2 for iv in intervals))
    if σ < 0.010 and μ < 1.0:
        return True, f"timing too regular (σ={σ*1000:.1f}ms, μ={μ*1000:.1f}ms)"
    return False, ""
```

Target: scripted attackers using `time.sleep(constant)` between requests. Empirically, σ for human browser navigation is >100 ms.

6 Proof-of-Work

For state-changing methods (`POST/PUT/DELETE`) and any path in `POW_REQUIRED_PATHS` = {/login, /api/auth, /admin}, the proxy requires a valid hashcash-style PoW solution. Without one, it returns `402 Payment Required` with a fresh challenge.

Challenge format

```
nonce | issued_ts | difficulty | HMAC-SHA256(nonce|issued_ts|difficulty, server_key)
↓
e31bd4c6bbaedb37 | 1777241780 | 5 | 03ca5327…0098b1b582d511e9231
```

Solution

The client must find `X` such that:

```
SHA256(nonce + X) starts with N hex zeros (default N=5)
```

With N=5, finding X requires ~10^5 hashes (≈ 0.3 s on modern CPU). The signed HMAC prevents challenge forgery; the 300-second TTL prevents replay over time.

Submission

```
POST /login HTTP/1.1
X-PoW-Token:    e31bd4c6…0098b1b582d511e9231
X-PoW-Solution: 844737
Content-Type:   application/x-www-form-urlencoded

user=admin&password=…
```

Verification

```
def verify_pow(token, solution):
    nonce, issued, diff, sig = token.split("|")
    payload = f"{nonce}|{issued}|{diff}"
    if not hmac.compare_digest(sig,
        hmac.new(KEY, payload.encode(), sha256).hexdigest()):
        return False, "bad signature"
    if int(time.time()) - int(issued) > 300:
        return False, "expired"
    h = sha256(f"{nonce}{solution}".encode()).hexdigest()
    if not h.startswith("0" * int(diff)):
        return False, f"hash {h[:8]} does not start with {diff} zeros"
    return True, "ok"
```

4. Configuration

4.1 Environment variables

Variable	Default	Effect
`UPSTREAM`	https://sporting.cfsectraining.com	Backend URL to forward to.
`LISTEN_HOST`	127.0.0.1	Bind address.
`LISTEN_PORT`	8443	Bind port.
`BURST`	5	Token-bucket burst capacity.
`REFILL`	1.0	Token refill rate (per second).
`TRUST_XFF`	first	`first` | `last` | `none` — which X-Forwarded-For entry to honour for IP resolution.

4.2 Internal endpoints (bypass all checks)

Path	Method	Purpose
`/__status`	GET	JSON snapshot of the per-IP state and config.
`/__pow`	GET	Issue a fresh PoW challenge.
`/__solver`	GET	Browser-side JavaScript PoW solver UI.
`/__xff`	GET	Debug — echoes received `X-Forwarded-For` chain and computed IP.

5. Test Results

All six layers were validated end-to-end against the live upstream and through the ngrok tunnel. Results:

Test	Expected	Observed	Status
Browser UA `Mozilla/5.0…`	200 forwarded	200	PASS
UA `curl/8.4.0`	403 ua-blocked	403 matched= `curl/`	PASS
UA `ClaudeBot`	403 ua-blocked	403 matched= `claude`	PASS
UA `python-requests`	403 ua-blocked	403 matched= `python-requests/`	PASS
GET `/wp-admin`	403 honeypot + 1h ban	403, then ban applied	PASS
Subsequent GET after ban	403 banned	403 banned for 3599s	PASS
Burst test (BURST=5, send 8)	1× 200, 7× 429	1× 200, 7× 429 Retry-After=1	PASS
POST `/login` w/o PoW	402 + challenge	402 challenge issued	PASS
POST w/ invalid PoW solution	402 + reason	402 "hash X does not start with 5 zeros"	PASS
PoW solve (difficulty=5)	~0.3s	0.34s, 641,466 attempts	PASS
POST `/login` w/ valid PoW	200 forwarded	200	PASS
12 reqs at ~150 ms regular	req 1-7 OK, 8+ blocked	req 8+ 403 σ=1.7 ms, μ=152 ms	PASS

6. Bypass Surface

By design, every layer has a known weakness so the proxy can be used as a testing harness. Each weakness reflects a real-world failure mode.

Layer	Weakness	Bypass technique	Severity
UA blocklist	Substring match on client-controlled header	Set `User-Agent: Mozilla/5.0…`	Trivial
Honeypot	Hard-coded path list, IP-keyed ban	Avoid the paths; rotate IPs to escape ban	Trivial
Rate limit	Bucket keyed on client-controlled IP (XFF)	Spoof `X-Forwarded-For` per request	High
Behavioral	Window of only 8 requests, σ threshold 10 ms	Inject jitter `sleep(uniform(0.05, 0.4))`	Trivial
PoW	Constant difficulty, replay window 5 min	Single solve serves multiple requests within 5 min	Medium
Internal endpoints	`/__*` paths bypass every check	Probe `/__status` to enumerate state	Medium

> **Key finding — ngrok-free trust boundary:** ngrok's free tier *passes through* the client-supplied `X-Forwarded-For` header without sanitisation. Setting `TRUST_XFF=last` does not prevent spoofing because the proxy sees only the value the client provided. Production deployments need a tier that overwrites or appends-only XFF (paid ngrok, Cloudflare Tunnel, or a trusted L7 LB with `X-Forwarded-For` normalisation).

7. Recommendations

The proxy demonstrates the techniques but should be hardened before any production-style use:

1. **IP resolution:** Replace `X-Forwarded-For` parsing with a trusted-proxy-aware resolver (`aiohttp_remotes.XForwardedStrict`) or terminate TLS behind a trusted edge that overwrites the chain.
2. **PoW replay protection:** Track used `(token, solution)` pairs server-side; bind challenges to method+path so a solved `/login` challenge can't authorise `/admin`.
3. **Adaptive PoW difficulty:** Increase difficulty exponentially under sustained load per IP (e.g. 5 → 6 → 7 zeros).
4. **Behavioral signals:** Move beyond timing; check for realistic HTTP/2 fingerprints, TLS JA4, accept-language presence, static-asset fetch sequences.
5. **Persistent state:** Move `ip_state` from in-process dict to Redis or DragonflyDB so restarts don't reset bans and multiple instances share state.
6. **Honeypot expansion:** Generate honeypots dynamically (random unique paths injected into HTML as zero-size links); attackers can no longer pre-list and avoid them.
7. **Edge integration:** For real protection, place Cloudflare Turnstile or hCaptcha in front of `POST` endpoints rather than relying on PoW alone — Turnstile in invisible mode adds bot-detection signals server-side PoW cannot reach.

8. Appendix

8.1 File inventory

File	Purpose	LOC
`proxy.py`	Main reverse proxy with all 6 layers	~290
`mock_upstream.py`	Echo backend on :9090 for isolated tests	~5
`test_final.py`	End-to-end integration test	~70
`test_behavior.py`	Behavioral-check timing test	~30
`README.md`	Usage and bypass documentation	~190

8.2 Run

```
cd /media/share/shared\ with\ kali-claude-code/anti-bot-proxy/

# Default (in front of sporting.cfsectraining.com, BURST=5, REFILL=1/s)
python3 proxy.py

# Custom config
TRUST_XFF=last BURST=100 REFILL=10.0 python3 proxy.py

# Public exposure via ngrok
ngrok http 8443
```

8.3 Wire example — full PoW exchange

```
# 1. Client posts without PoW headers
$ curl -X POST -H 'User-Agent: Mozilla/5.0' \
       -d 'user=admin' https://<tunnel>/login

HTTP/2 402 Payment Required
X-Reason: pow-required
X-PoW-Challenge: e31b…|1777241780|5|03ca…
X-PoW-Difficulty: 5
Content-Type: application/json

{"error":"Proof-of-Work required",
 "challenge":"e31b…|1777241780|5|03ca…",
 "difficulty":5}

# 2. Client solves
nonce = "e31b…"
for i in range(10**8):
    if sha256(f"{nonce}{i}".encode()).hexdigest().startswith("00000"):
        solution = str(i)
        break

# 3. Client retries with PoW headers
$ curl -X POST -H 'User-Agent: Mozilla/5.0' \
       -H 'X-PoW-Token: e31b…|1777241780|5|03ca…' \
       -H 'X-PoW-Solution: 844737' \
       -d 'user=admin' https://<tunnel>/login

HTTP/2 200 OK
... (forwarded upstream response)
```

8.4 State snapshot

```
$ curl -s http://127.0.0.1:8443/__status | jq
{
  "clients": {
    "94.61.192.8": {
      "tokens": 100,
      "request_count": 7,
      "banned_until": 0,
      "first_seen_secs_ago": 12.4
    }
  },
  "config": {
    "burst": 100,
    "refill_per_sec": 10.0,
    "pow_difficulty": 5,
    "honeypot_ban_secs": 3600
  }
}
```

End of report — AntiBot-GW/1.0 Implementation Report — Pedro Tarrinho — 2026-04-26